Урок 04

Контейнеры STL –
std::vector, std::array, std::string

Что сегодня изучаем?

- что такое контейнеры STL и зачем они нужны;
- работа с фиксированным массивом **std::array**;
- динамические массивы и методы **std::vector**;
- хранение и обработка строк через **std::string**;
- сравнение контейнеров и перебор элементов циклом **for**;
- практика: обработка коллекций и текстовых данных.

Что такое std::array?

std::array — тот же самый массив, но с методами для более удобной работы

Важно
объявление и инициализация →

```cpp
#include <array>

array <int, 5> arr = {1, 2, 3, 4, 5};
```

Рисунок 1 — объявления и инициализация массива

Основные методы для работы с std::array

```
cout << arr.size();  // 5
cout << arr.front(); // 1
cout << arr.back();  // 5
arr.fill(10);        // все элементы = 10
cout << arr.at(2);   // безопасный доступ
```

Рисунок 2 — применение методов

- **size()** - возвращает количество элементов в массиве;
- **front()** - возвращает первый элемент из массива;
- **back()** - возвращает последний элемент из массива;
- **fill(значение)** - заполняет все элементы заданным значением;
- **at(2)** - возвращает значение элемента массива по его индексу

Что такое std::vector?

std::vector — динамический массив

- Размер может меняться во время выполнения;
- Динамический массив с автоматическим расширением;
- Очень гибкий, чаще всего используется в проектах.

Важно
объявление и инициализация

```
#include <string>

vector<int> vec = {1, 2, 3}
```

Рисунок 3 — объявление и инициализация вектора

Основные методы для работы с std::vector

```
v.push_back(4);    // добавить элемент
v.pop_back();      // удалить последний элемент
cout << v.size(); // размер
cout << v[0];      // доступ к элементу
v.clear();         // очистка
```

Рисунок 4 — основные методы

- **push_back(значение)** - добавляет указанное значение в конец вектора;
- **pop_back()** - удаляет последнее значение;
- **size()** - возвращает количество элементов в векторе;
- **clear()** - очищает вектор (удаляет все значения из него).

Что такое std::string?

std::string — символьный массив, который выглядит как строка

Важно
объявление и инициализация

```
#include <string>

string s = "Hello world!"
```

Рисунок 5 — объявление и инициализация строки

Примечание
std::string это почти то же самое, что и std::vector<char>, только с расширенным функционалом

Основные методы для работы с std::string

```
cout << s.size();         // длина
cout << s[0];             // первый символ
cout += "!!!";            // конкатенация
cout << s.substr(0, 5);   // "Hello"
cout << s.find("world"); // позиция строки
```

Рисунок 6 — основные методы

- **substr(значение1, значение2)** - выводит часть строки по указанным индексам;
- **find(значение)** - возвращает индекс начала подстроки;
- **size()** - возвращает количество символов в строке.

Сравнительная характеристика

Тип	Размер	Методы	Примеры использования
int arr[10]	фиксирован	нет	низкоуровневый код, драйверы
std::array	фиксирован	есть	фиксированные таблицы, матрицы
std::vector	динамический	есть	прикладное ПО, алгоритмы
std::string	динамический	есть	любая работа с текстом

for-each

```
for (тип_элемента переменная : контейнер){
  // тело цикла
}
```

Рисунок 7 — синтаксис range based

for-each - цикл для перебора элементов в контейнерах

Практическое задание

Задание 1

Создайте программу для обработки оценок студентов:

- сохраните 5 оценок в std::array;
- выведите первую, последнюю и общее количество оценок;
- скопируйте оценки в std::vector;
- добавьте две оценки с помощью **push_back();**
- вычислите среднее арифметическое всех оценок.



Практическое задание

Задание 2

Создайте программу для работы со строкой:

- запросите у пользователя имя и фамилию;
- объедините их в одну строку **std::string**;
- определите длину полученной строки;
- найдите позицию пробела с помощью **find()**;
- выделите имя с помощью s**ubstr()**;
- выведите результат на экран.



Вывод

За сегодня вы узнали:

- **std::array** хранит фиксированное количество элементов;
- **std::vector** позволяет динамически изменять количество элементов;
- **std::string** предназначен для удобной работы с текстом;
- **range-based for** упрощает перебор элементов контейнеров.

Источники:

- LearnCpp — Introduction to containers and arrays
- cppreference — std::array
- cppreference — std::vector
- cppreference — std::basic_string

Вспомогательные вопросы

1. Что такое контейнеры STL?
2. Чем std::array отличается от обычного массива?
3. Почему размер std::array нельзя изменить?
4. Чем std::vector отличается от std::array?
5. Для чего используются push_back() и pop_back()?
6. Какие задачи решает std::string?
7. Чем отличаются методы find() и substr()?
8. Как перебрать элементы контейнера с помощью range-based for?

Домашнее задание

1. Сделать конспект;
2. Ответить на вспомогательные вопросы;
3. Доработать практическое задание

